

Setup: install what you'll need

```
cd ~/secure-webapp-capstone/app
source venv/bin/activate
pip install bcrypt flask-talisman flask-wtf flask-limiter
```

Fix 1: SQL Injection

Open the file:

```
bash
nano app.py
```

Find this in the `search()` function:

```
python
sql = "SELECT id, name, department FROM employees WHERE name LIKE '%" + query + "%"
cur.execute(sql)
```

Replace with:

```
python
cur.execute("SELECT id, name, department FROM employees WHERE name LIKE ?", (f"%{query}%",))
```

Save (Ctrl+O, Enter, Ctrl+X).

Re-test:

```
bash
python3 app.py
```

Visit <http://127.0.0.1:5000/search?q=' OR '1'='1> → should now show **zero results**, not all 4 employees.

Fix 2: Admin panel access control

In `app.py`, find:

```
python
@app.route('/admin')
def admin():
    # Intentional vuln: no role check - any logged-in user can access
    if 'user_id' not in session:
        return redirect('/login')
```

Replace with:

```
python
@app.route('/admin')
```

```
def admin():
    if 'user_id' not in session:
        return redirect('/login')
    if session.get('role') != 'admin':
        return "Forbidden - Admins only", 403
```

Re-test: log in as Alice, visit `/admin` → should show **403 Forbidden**.

Fix 3: IDOR on profile

Find:

```
python
@app.route('/profile/<int:emp_id>', methods=['GET', 'POST'])
def profile(emp_id):
    if 'user_id' not in session:
        return redirect('/login')
    db = get_db()
    cur = db.cursor()

    if request.method == 'POST':
        # Intentional vuln: no check that session user_id == emp_id (IDOR)
        new_bio = request.form['bio']
```

Replace with:

```
python
@app.route('/profile/<int:emp_id>', methods=['GET', 'POST'])
def profile(emp_id):
    if 'user_id' not in session:
        return redirect('/login')
    if session['user_id'] != emp_id and session.get('role') != 'admin':
        return "Forbidden - you can only view your own profile", 403
    db = get_db()
    cur = db.cursor()

    if request.method == 'POST':
        new_bio = request.form['bio']
```

Re-test: as Alice, visit `/profile/2` → should show **403 Forbidden**. `/profile/1` (her own) should still work.

Fix 4: Stored XSS

```
bash
nano templates/profile.html
```

Find:

html

```
<p>Bio: {{ emp[4]|safe }}</p>
```

Replace with:

```
html
<p>Bio: {{ emp[4] }}</p>
```

(Removing `|safe` restores Jinja2's automatic escaping.)

Re-test: save `<script>alert('XSS')</script>` as bio → it should now display as literal text on the page, no popup.

Fix 5: Broken Authentication — password hashing

5a. Update `init_db.py` to hash passwords when creating sample data:

```
bash
nano init_db.py
```

At the top, add:

```
python
import bcrypt
```

Find:

```
python
employees = [
    ('Alice Sharma', 'alice@corp.com', 'alice123', 'employee', 'Engineering', 'Loves Python and coffee.'),
    ('Ravi Kumar', 'ravi@corp.com', 'ravi456', 'employee', 'HR', 'HR generalist, 5 years experience.'),
    ('Priya Nair', 'priya@corp.com', 'priya789', 'employee', 'Finance', 'Finance analyst.'),
    ('Admin User', 'admin@corp.com', 'admin123', 'admin', 'IT', 'System administrator.'),
]
```

Replace with:

```
python
def hash_pw(p):
    return bcrypt.hashpw(p.encode(), bcrypt.gensalt()).decode()

employees = [
    ('Alice Sharma', 'alice@corp.com', hash_pw('alice123'), 'employee', 'Engineering', 'Loves Python and
coffee.'),
    ('Ravi Kumar', 'ravi@corp.com', hash_pw('ravi456'), 'employee', 'HR', 'HR generalist, 5 years
experience.'),
    ('Priya Nair', 'priya@corp.com', hash_pw('priya789'), 'employee', 'Finance', 'Finance analyst.'),
    ('Admin User', 'admin@corp.com', hash_pw('admin123'), 'admin', 'IT', 'System administrator.'),
]
```

Re-initialize the database (delete the old plaintext one first):

```
bash
rm portal.db
python3 init_db.py
```

5b. Update `app.py` login logic and add rate limiting:

```
bash
nano app.py
```

At the top, add imports:

```
python
import bcrypt
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
```

After `app = Flask(__name__)`, add:

```
python
limiter = Limiter(get_remote_address, app=app, default_limits=[])
```

Find the login function:

```
python
@app.route('/login', methods=['GET', 'POST'])
def login():
    error = None
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        db = get_db()
        cur = db.cursor()
        # Intentional vuln: plaintext password comparison, no rate limiting
        cur.execute("SELECT id, name, role FROM employees WHERE email=? AND password=?", (email,
password))
        user = cur.fetchone()
        if user:
```

Replace with:

```
python
@app.route('/login', methods=['GET', 'POST'])
@limiter.limit("5 per minute")
def login():
    error = None
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        db = get_db()
        cur = db.cursor()
        cur.execute("SELECT id, name, role, password FROM employees WHERE email=?", (email,))
        user = cur.fetchone()
        if user and bcrypt.checkpw(password.encode(), user[3].encode()):
```

Below that, the line assigning session values stays the same (`user[0], user[1], user[2]`) — just make sure it still reads `user[0], user[1], user[2]` for id/name/role, not the password field.

Re-test:

```
bash
sqlite3 portal.db "SELECT email, password FROM employees;"
```

Passwords should now show long bcrypt hashes (starting with `$2b$ . . .`), not plaintext.

Try logging in wrong 6 times quickly — the 6th attempt should get blocked with a "429 Too Many Requests" response.

6a. Security headers with Talisman. In `app.py`, add near the top:

```
python
from flask_talisman import Talisman
```

After `app = Flask(__name__)`, add:

```
python
Talisman(app, content_security_policy={
    'default-src': "'self'",
},
```

6b) **CSRF protection.** Add:
`nano app.py`

Find these lines near the top:

```
python
app = Flask(__name__)
limiter = Limiter(get_remote_address, app=app, default_limits=[])
Talisman(app, content_security_policy={
    'default-src': "'self'",
}, force_https=False)
app.secret_key = "supersecretkey123" # weak, hardcoded (intentional vuln)
```

Add the CSRF initialization line right after:

```
python
app = Flask(__name__)
limiter = Limiter(get_remote_address, app=app, default_limits=[])
Talisman(app, content_security_policy={
    'default-src': "'self'",
}, force_https=False)
csrf = CSRFProtect(app)
app.secret_key = "supersecretkey123" # weak, hardcoded (intentional vuln)
```

Save (Ctrl+O, Enter, Ctrl+X), then restart:

```
bash
python3 app.py
```

Go to `http://127.0.0.1:5000/login` again — the page should load fine now instead of throwing a 500 error.

One more thing to double check while you're at it — did you already re-run `init_db.py` with the bcrypt hashing update? Run this to confirm:

```
bash
sqlite3 portal.db "SELECT password FROM employees LIMIT 1;"
```

If it shows plaintext like `alice123` instead of a hash starting with `$2b$`, run:

```
bash
rm portal.db
python3 init_db.py
```

6c. Disable debug mode / suppress version disclosure. At the bottom of `app.py`, find:

```
python
if __name__ == '__main__':
    # Intentional vuln: debug=True exposes stack traces in production
    app.run(debug=True, host='0.0.0.0', port=5000)
```

Replace with:

```
python
if __name__ == '__main__':
    app.run(debug=False, host='0.0.0.0', port=5000)
```

Re-test: re-run your ZAP automated scan — the CSP, X-Frame-Options, X-Content-Type-Options, and CSRF alerts should be gone or reduced.

Restart and verify everything together

```
bash
python3 app.py
```